

Collegium Witelona Uczelnia Państwowa w Legnicy
Wydział Nauk Technicznych i Ekonomicznych
Kierunek: Informatyka



**Projekt z przedmiotu "Projektowanie i programowanie systemów
internetowych II"**

Temat: "Strona dla miłośników monstera".

Autorzy

Mateusz Bogacz-Drewniak, nr. indeksu: 44491

Mateusz Chimkowski, nr. indeksu: 43831

Szymon Mikołajek, nr. indeksu: 41105

Paweł Kruk, nr. indeksu: 43854

Bartosz Dobroś, nr. indexu: 41053

Prowadzący przedmiot
mgr inż. Zdzisław Pawelec

Legnica, 2026

Spis treści

1	Cel i ogólna charakterystyka projektu	3
1.1	Wykaz funkcjonalności	3
1.1.1	Moduł Autoryzacji i Zarządzania Tożsamością Użytkownika	3
1.1.2	Moduł Katalogowy i Partycypacji Społecznościowej	3
1.1.3	Moduł Sztucznej Inteligencji i Systemów Eksperckich	3
1.1.4	Moduł Harmonogramowania i Zarządzania Czasem	4
1.1.5	Panel Administracyjny i Utrzymanie Systemu	4
1.2	Charakterystyka systemu	5
1.3	Najważniejsze założenia	5
2	Technologie i wymagania	6
2.1	Wykorzystane technologie	6
2.1.1	Backend	6
2.1.2	Frontend	6
2.1.3	Stos technologiczny	6
3	Architektura i przepływ danych	7
3.1	Model relacyjny bazy danych (ERD)	7
3.2	Przepływ danych w systemie (DFD)	8
4	Instrukcja uruchomienia systemu	8
4.1	Wymagania systemowe	8
4.2	Procedura uruchomienia warstwy Backend	9
4.3	Procedura uruchomienia warstwy Frontend	9
5	Implementacja	9
5.1	Główny framework aplikacyjny (Python i FastAPI)	9
5.2	Wizja Komputerowa i Sieci Neuronowe (PyTorch i TorchVision)	10
5.3	Optymalizacja Harmonogramów (Natywny Algorytm Genetyczny)	11
5.4	System Rekomendacyjny (Drzewo Decyzyjne)	12
6	Kluczowe fragmenty kodu źródłowego	14
6.1	Implementacja Uczenia Transferowego (Transfer Learning)	14
6.2	Dwuetapowa inferencja modelu wizyjnego w API	15
6.3	Funkcja oceny (Fitness) w Algorytmie Genetycznym	16
6.4	Mutacja z wykorzystaniem rozkładu Gaussa	16
6.5	Silnik wnioskujący Systemu Eksperckiego	17
7	Prezentacja interfejsu systemu	18
7.1	Aplikacja Frontend (Widok Użytkownika)	18
7.2	Aplikacja Backend (Dokumentacja API Swagger)	23
8	Podsumowanie i wnioski ogólne	24

9	Wkład, udział i wnioski osobiste	25
9.1	Mateusz Bogacz-Drewniak	25
9.2	Mateusz Chimkowski	25
9.3	Szymon Mikołajek	25
9.4	Paweł Kruk	25
9.5	Bartosz Dobroś	25

1 Cel i ogólna charakterystyka projektu

MonsterAI to zintegrowany system, który łączy rozpoznawanie wizualne puszek, listę wypitych napojów, dopasowanie preferencji użytkownika oraz planowanie spożycia napojów energetycznych "Monster" w ciągu dnia.

1.1 Wykaz funkcjonalności

1.1.1 Moduł Autoryzacji i Zarządzania Tożsamością Użytkownika

- Rejestracja i profilowanie: System umożliwia utworzenie konta wraz z jednoczesnym zdefiniowaniem preferencji smakowych, obejmujących m.in. tolerancję na cukier oraz profilowanie w kierunku smaków słodkich, kwaśnych lub umiarkowanych.
- Weryfikacja tożsamości: Wdrożono proces asynchronicznej wysyłki wiadomości e-mail z tokenem weryfikacyjnym, wymagany do aktywacji nowo utworzonego konta w systemie.
- Zarządzanie sesją: Bezpieczeństwo uwierzytelniania oparto na standardzie JSON Web Token (JWT), implementując mechanizm krótkoterminowego tokenu dostępu (Access Token) oraz tokenu odświeżającego (Refresh Token).
- Procedura odzyskiwania dostępu: Zaimplementowano bezpieczny proces resetowania hasła, inicjowany poprzez asynchroniczną wysyłkę dedykowanego tokena autoryzacyjnego na adres e-mail powiązany z kontem.

1.1.2 Moduł Katalogowy i Partycypacji Społecznościowej

- Prezentacja danych: Aplikacja udostępnia interfejs do przeglądania bazy napojów energetycznych, wykorzystujący optymalizację przesyłu danych poprzez mechanizmy stronicowania (paginacji) oraz dynamicznego sortowania.
- Ewidencja zasobów i konsumpcji: Użytkownicy posiadają możliwość śledzenia aktywności poprzez oznaczanie produktów jako skonsumowanych oraz tworzenia wirtualnego inwentarza fizycznie posiadanych jednostek.
- System ocen i metadane: Zaimplementowano możliwość parametryzacji produktów poprzez wystawianie ocen numerycznych oraz dodawanie prywatnych notatek tekstowych przez użytkowników.
- Agregacja danych społecznościowych: Logika systemu na bieżąco przetwarza dane wejściowe od społeczności, obliczając i aktualizując średnią ocenę dla poszczególnych produktów w bazie.

1.1.3 Moduł Sztucznej Inteligencji i Systemów Eksperckich

- Detekcja i klasyfikacja obrazu (Computer Vision): Wdrożono dwuetapowy proces inferencji oparty na modelach sieci neuronowych implementowanych w bibliotece PyTorch. Pierwszy model (binarny) weryfikuje obecność obiektu docelowego na obrazie, po czym drugi model dokonuje klasyfikacji wieloklasowej konkretnego wariantu napoju, zwracając wyniki wraz z miarą pewności (confidence).

- Optymalizacja wielokryterialna (Algorytmy Genetyczne): Silnik generowania harmonogramów wykorzystuje algorytm genetyczny w celu optymalizacji dystrybucji dawek kofeiny. Algorytm inicjuje populację rozwiązań i stosuje operatory selekcji, krzyżowania oraz mutacji, dążąc do maksymalizacji pokrycia zaplanowanych zadań przy jednoczesnej minimalizacji nakładania się efektów stymulujących oraz uwzględnieniu okien snu i czuwania.
- System Rekomendacyjny: Zastosowano logikę drzewa decyzyjnego (reprezentowaną strukturą JSON) do filtrowania i personalizowania propozycji produktów na podstawie wejściowych wektorów cech z ankiety, uwzględniających budżet, preferowane kanały dystrybucji i profil smakowy.

1.1.4 Moduł Harmonogramowania i Zarządzania Czasem

- Trwałość danych (Persystencja): Zaimplementowano mechanizm zapisu wygenerowanych algorytmicznie planów dnia, umożliwiającą wgląd w historyczne iteracje harmonogramów.
- Strukturyzacja procesów: Zapisane obiekty przechowują szczegółowe informacje dotyczące wyliczonych interwałów czasowych (sesji konsumpcji) oraz przypisanych do nich konkretnych zadań użytkownika.
- Cykl życia obiektu: Architektura udostępnia endpointy pozwalające na trwałe usuwanie zdezaktualizowanych planów, zabezpieczone weryfikacją uprawnień właścicielskich.

1.1.5 Panel Administracyjny i Utrzymanie Systemu

- Zarządzanie zasobami (CRUD): Administratorzy posiadają narzędzia do pełnej modyfikacji bazy asortymentowej, w tym definiowania zawartości substancji aktywnych (kofeiny) oraz kategoryzowania kanałów dystrybucji.
- Integracja z chmurą obliczeniową: System w sposób asynchroniczny zarządza zasobami statycznymi (zdjęciami), wykorzystując integrację z zewnętrzną obiektową pamięcią masową zgodną z protokołem S3.
- Polityka retencji danych (Soft Delete): Proces usuwania rekordów zrealizowano jako operację logiczną ("miękkie usuwanie") zabezpieczającą spójność historyczną i analityczną bazy danych, połączoną z jednoczesnym fizycznym usunięciem zasobów graficznych z zewnętrznego serwera S3.
- Kontrola dostępu i uprawnień: System wyposażono w mechanizmy blokowania i odblokowywania użytkowników standardowych. Ponadto, zarządzanie kontami administracyjnymi objęto restrykcjami strukturalnymi, zapobiegającymi krytycznym błędom, takim jak usunięcie własnego profilu lub dezaktywacja ostatniego administratora w systemie.

1.2 Charakterystyka systemu

System ma charakter narzędzia wspomagającego dobór, klasyfikację oraz planowanie spożycia napojów energetycznych. Nie opiera się na ogólnym, trudnym do kontrolowania wnioskowaniu, lecz wykorzystuje zestaw ukierunkowanych metod sztucznej inteligencji: drzewa decyzyjne do mapowania preferencji, dedykowane modele wizji komputerowej do rozpoznawania obrazu oraz algorytmy genetyczne do optymalizacji czasu konsumpcji. Dzięki temu system działa wydajnie (czas reakcji poniżej 10 sekund), ściśle przestrzega twardych ograniczeń fizjologicznych (takich jak higiena snu czy limity kofeiny) i pozwala na generowanie spersonalizowanych, powtarzalnych oraz bezpiecznych wyników.

1.3 Najważniejsze założenia

Najważniejsze założenia projektowe są następujące:

- wejściem dla modułu rozpoznawania wizualnego jest pojedynczy plik graficzny (zdjęcie puszki wgrane z galerii lub zrobione aparatem telefonu),
- komunikacja między interfejsem użytkownika a backendem, w tym wyniki działania poszczególnych modułów, realizowana jest w ustandaryzowanym formacie JSON,
- reguły wnioskowania dla drzewa decyzyjnego (quizu) oraz definicje klas dla modeli wizyjnych (AI Vision) są wczytywane z zewnętrznych plików tekstowych (np. `tree.json`, `classes.json`), co pozwala na modyfikację logiki rekomendacji bez konieczności ingerencji w kod źródłowy aplikacji,
- system odciąża serwer główny poprzez asynchroniczne przesyłanie plików multimedialnych do zewnętrznej chmury obiektywnej (S3) zamiast przechowywać je lokalnie,
- baza danych wykorzystuje mechanizm „miękkiego usuwania” (soft delete), zachowując integralność historyczną danych użytkowników, a z systemu trwale usuwane są jedynie obciążające zasoby pliki graficzne,
- wydajność algorytmów sztucznej inteligencji została zoptymalizowana w taki sposób, aby czas odpowiedzi modułów predykcyjnych i planujących (algorytm genetyczny) nie przekraczał 10 sekund.

2 Technologie i wymagania

2.1 Wykorzystane technologie

2.1.1 Backend

Kategoria	Technologia
Framework API	FastAPI
Serwer aplikacyjny	Uvicorn
Mapowanie bazy danych (ORM)	SQLModel / SQLAlchemy
Zarządzanie migracjami	Alembic
Sterownik bazy danych	psycpg2-binary
Sztuczna inteligencja (AI)	PyTorch
Przetwarzanie obrazu i danych	Pillow / NumPy
Walidacja danych	Pydantic
Bezpieczeństwo i autoryzacja	PyJWT / bcrypt
Integracja z chmurą (Storage)	aioboto3 / boto3

Tabela 1: Technologie backendowe wykorzystane w projekcie

2.1.2 Frontend

Kategoria	Technologia
Główna Biblioteka UI	React
Routing i Framework	React Router
Framework CSS	Tailwind CSS
Narzędzie Budowania	Vite

Tabela 2: Technologie frontendowe wykorzystane w projekcie

2.1.3 Stos technologiczny

Kategoria	Technologia
Konteneryzacja	Docker
Orkiestracja Środowiska	Docker Compose
Zarządzanie Zależnościami	UV (Astral)
Pamięć Obiektowa (S3)	MinIO
Testowanie Email (SMTP)	Mailpit
Baza Danych	PostgreSQL

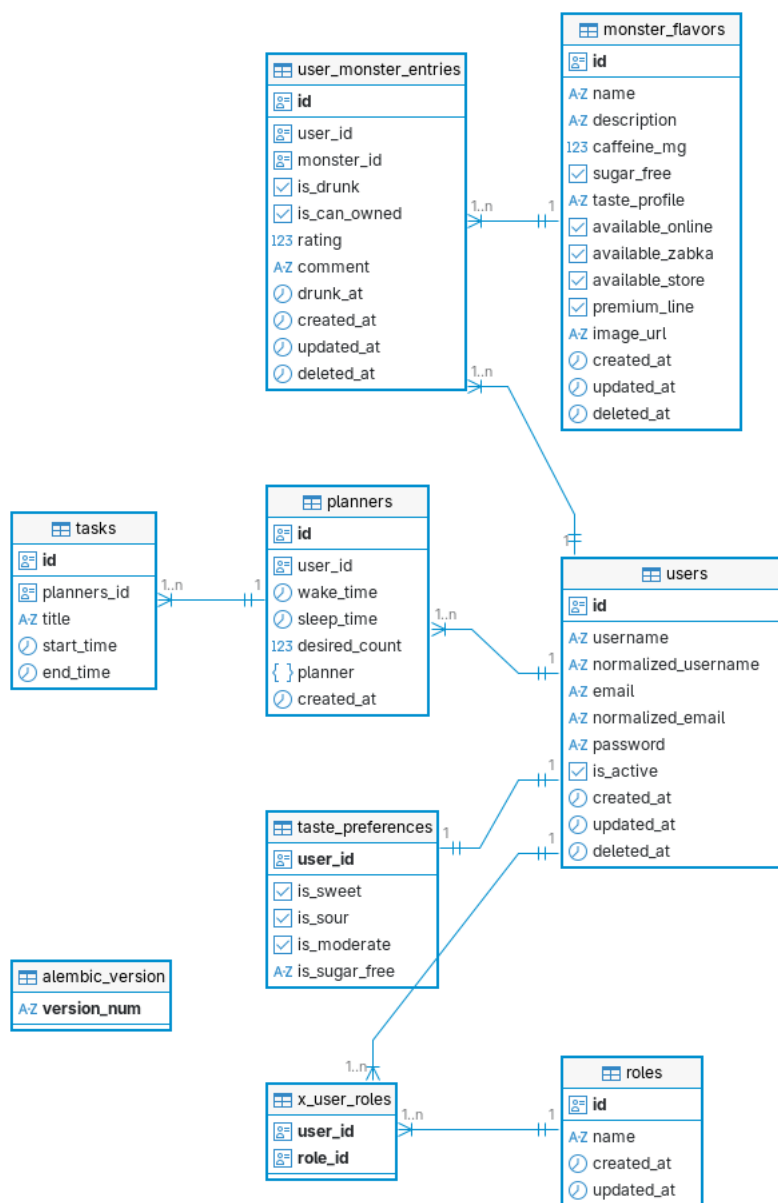
Tabela 3: Technologie infrastrukturalne wykorzystane w projekcie

3 Architektura i przepływ danych

W celu zapewnienia wysokiej skalowalności oraz modularności systemu MonsterAI, zaprojektowano architekturę opartą na separacji warstwy danych, logiki biznesowej i prezentacji.

3.1 Model relacyjny bazy danych (ERD)

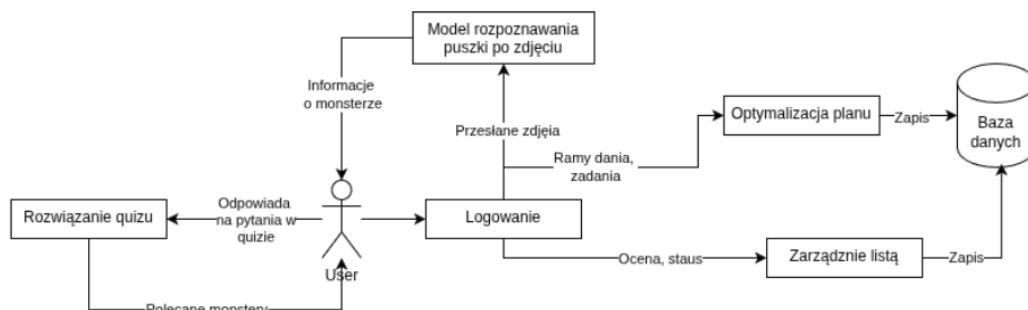
Struktura bazy danych została zoptymalizowana pod kątem przechowywania danych użytkowników, ich preferencji oraz historycznych harmonogramów spożycia kofeiny. Relacje obejmują m.in. powiązania między użytkownikami a ich osobistymi kolekcjami napojów oraz zadaniami w planerze.



Rysunek 1: Diagram związków encji (ERD) przedstawiający strukturę bazy danych systemu.

3.2 Przepływ danych w systemie (DFD)

Diagram przepływu danych obrazuje interakcje między użytkownikiem a poszczególnymi modułami sztucznej inteligencji oraz proces zapisu danych w persystentnej pamięci masowej (PostgreSQL i S3).



Rysunek 2: Diagram przepływu danych (DFD)

4 Instrukcja uruchomienia systemu

W celu weryfikacji funkcjonalności, system został przystosowany do uruchomienia w środowisku lokalnym przy wykorzystaniu technologii konteneryzacji.

4.1 Wymagania systemowe

Przed przystąpieniem do procedury uruchomienia, należy upewnić się, że w systemie zainstalowane są następujące narzędzia:

- **Docker Desktop** (wraz z wtyczką Docker Compose) – do orkiestracji usług backendowych.
- **Node.js** (wersja stabilna) oraz menedżer pakietów **npm** – do obsługi warstwy frontendowej.
- Narzędzie **git** – do klonowania repozytoriów.

4.2 Procedura uruchomienia warstwy Backend

Moduł serwerowy składa się z kilku powiązanych usług (API, PostgreSQL, MinIO, Mailpit). Proces ich inicjalizacji przebiega następująco:

1. Pobranie najnowszej wersji kodu źródłowego z oficjalnego repozytorium:
`https://github.com/Monster-IA-Team/Monster-IA-Backend`
2. Przejście do katalogu głównego projektu.
3. Konfiguracja środowiska: Należy utworzyć plik `.env` na podstawie dostarczonego wzorca `.env.example`. W sekcji `API_JWT_SECRET_KEY` należy umieścić wygenerowany klucz bezpieczeństwa dla standardu JWT.
4. Uruchomienie kontenerów poleceniem:
`docker compose up -build`.
5. Proces budowania i konfiguracji bazy (w tym migracje `alembic`) potrwa od 2 do 5 minut. Po tym czasie usługi będą dostępne.

4.3 Procedura uruchomienia warstwy Frontend

Aplikacja kliencka (React) uruchamiana jest niezależnie od kontenerów backendowych:

1. Pobranie kodu źródłowego frontendu:
`https://github.com/Monster-IA-Team/Monster-IA-Frontend`
2. Przejście do katalogu projektu w terminalu.
3. Instalacja niezbędnych zależności projektowych poleceniem:
`npm install`
4. Uruchomienie serwera deweloperskiego:
`npm run dev`

Po wykonaniu powyższych kroków, aplikacja webowa będzie dostępna pod adresem `http://localhost:5173`, natomiast dokumentacja interaktywna API (Swagger) pod adresem `http://localhost:8000/docs`. Szczegółowe informacje techniczne dotyczące konfiguracji poszczególnych modułów znajdują się w plikach `README.md` w odpowiednich repozytoriach.

5 Implementacja

5.1 Główny framework aplikacyjny (Python i FastAPI)

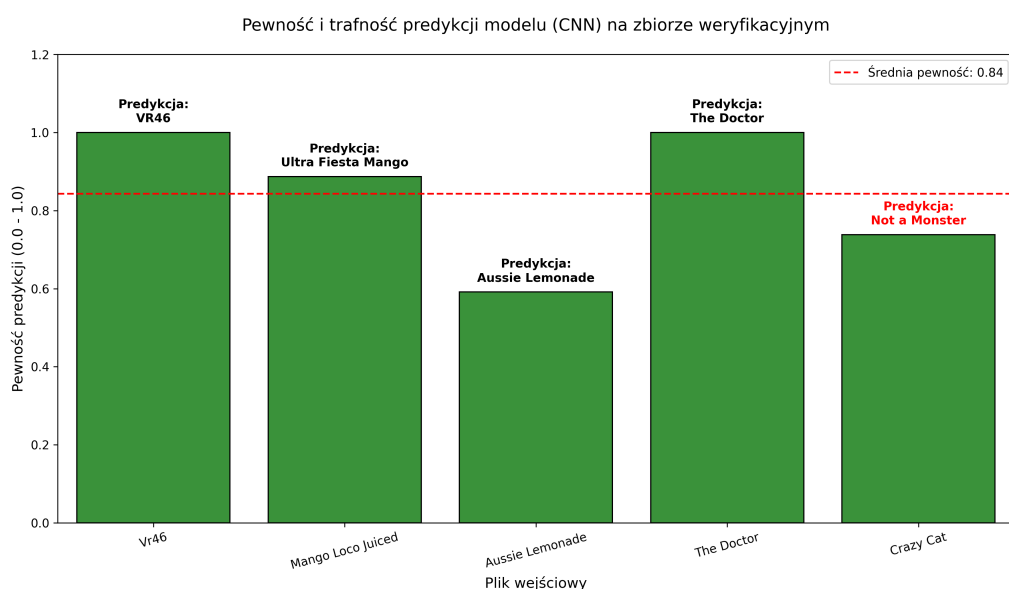
Jako fundament warstwy serwerowej wybrano język Python wraz z nowoczesnym frameworkiem FastAPI.

- **Uzasadnienie wyboru:** Python stanowi natywne środowisko dla wiodących bibliotek sztucznej inteligencji (takich jak PyTorch), co eliminuje konieczność tworzenia złożonej architektury mikroserwisowej. FastAPI wybrano ze względu na pełne wsparcie dla operacji asynchronicznych (kluczowe przy przetwarzaniu obrazów) oraz automatyczną walidację danych wejściowych opartą na standardzie Pydantic.

5.2 Wizja Komputerowa i Sieci Neuronowe (PyTorch i TorchVision)

Moduł rozpoznawania produktów (AI Vision) zrealizowano w oparciu o bibliotekę PyTorch. System wykorzystuje model binarny do detekcji puszek oraz klasyfikator wieloklasowy do rozpoznawania wariantu smakowego.

- **Transfer Learning:** Zamiast trenowania sieci od podstaw, zastosowano uczenie transferowe na architekturach ResNet18 oraz MobileNetV2. Wykorzystanie wag pre-trenowanych na zbiorze ImageNet pozwoliło uzyskać wysoką precyzję przy relatywnie małym zbiorze danych własnych. W procesie uczenia (optimizer Adam, CrossEntropyLoss) zmodyfikowano jedynie końcową warstwę w pełni połączoną (`nn.Linear`), dostosowując ją do liczby smaków w systemie.

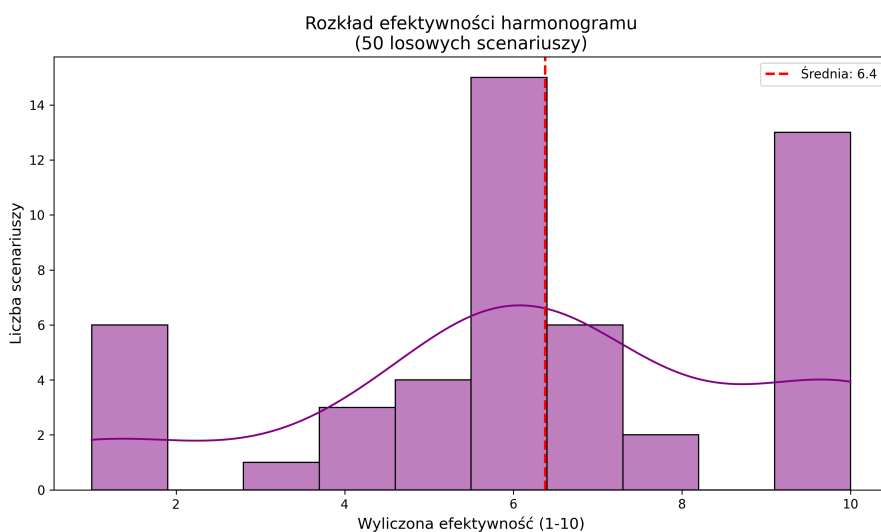


Rysunek 3: Pewność i trafność predykcji modelu wizyjnego na zbiorze weryfikacyjnym.

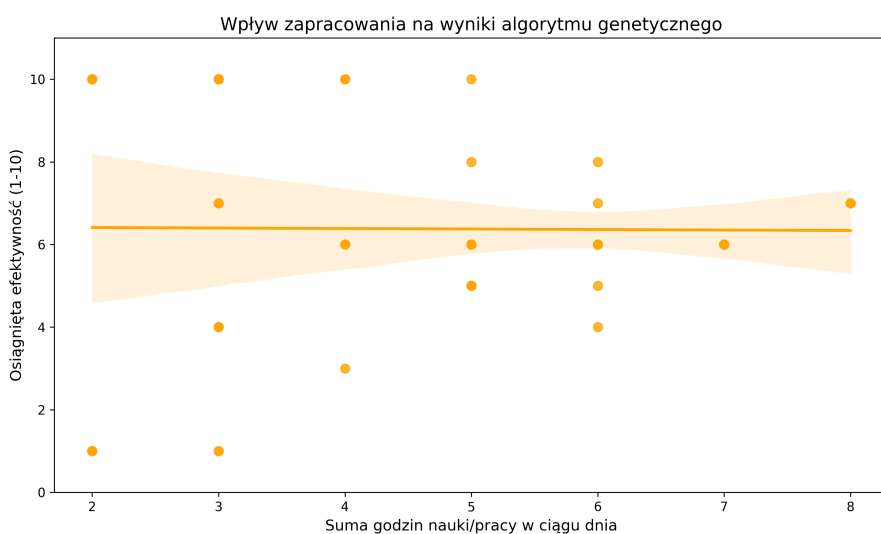
5.3 Optymalizacja Harmonogramów (Natywny Algorytm Genetyczny)

Silnik wyliczający optymalne godziny spożycia kofeiny opiera się na autorskiej implementacji Algorytmu Genetycznego w języku Python.

- **Logika ewolucyjna:** Zastosowano selekcję turniejową oraz mutację opartą o rozkład Gaussa, co pozwala na precyzyjne przeszukiwanie przestrzeni rozwiązań bez utraty stabilności. Funkcja Fitness uwzględnia kary za nakładanie się dawek (overlap penalty) oraz nagradza rozwiązania maksymalizujące wydajność w oknach czasowych zdefiniowanych zadań.



Rysunek 4: Histogram rozkładu funkcji Fitness wykazujący wysoką zbieżność algorytmu genetycznego.

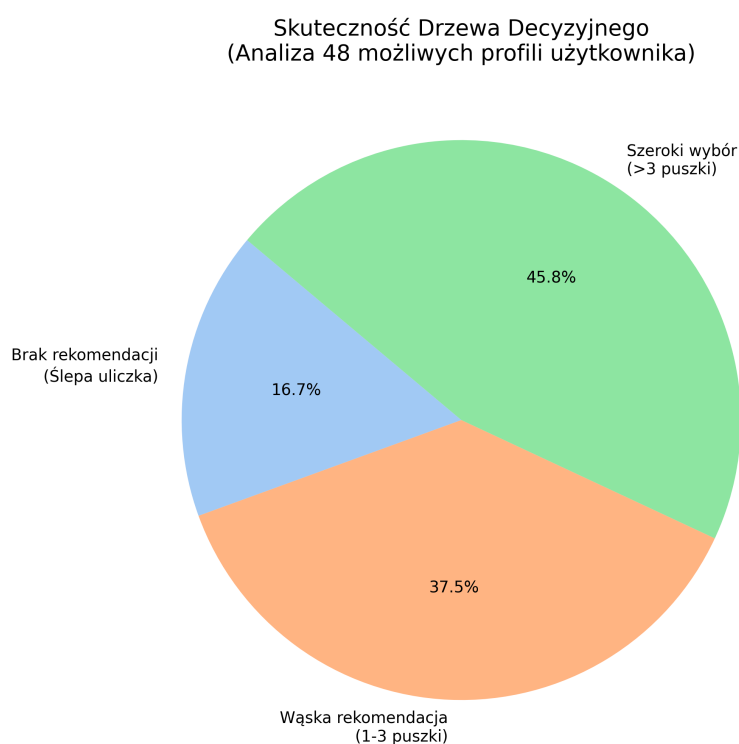


Rysunek 5: Diagram wpływu zapracowania (liczby zadań) na efektywność wygenerowanego planu.

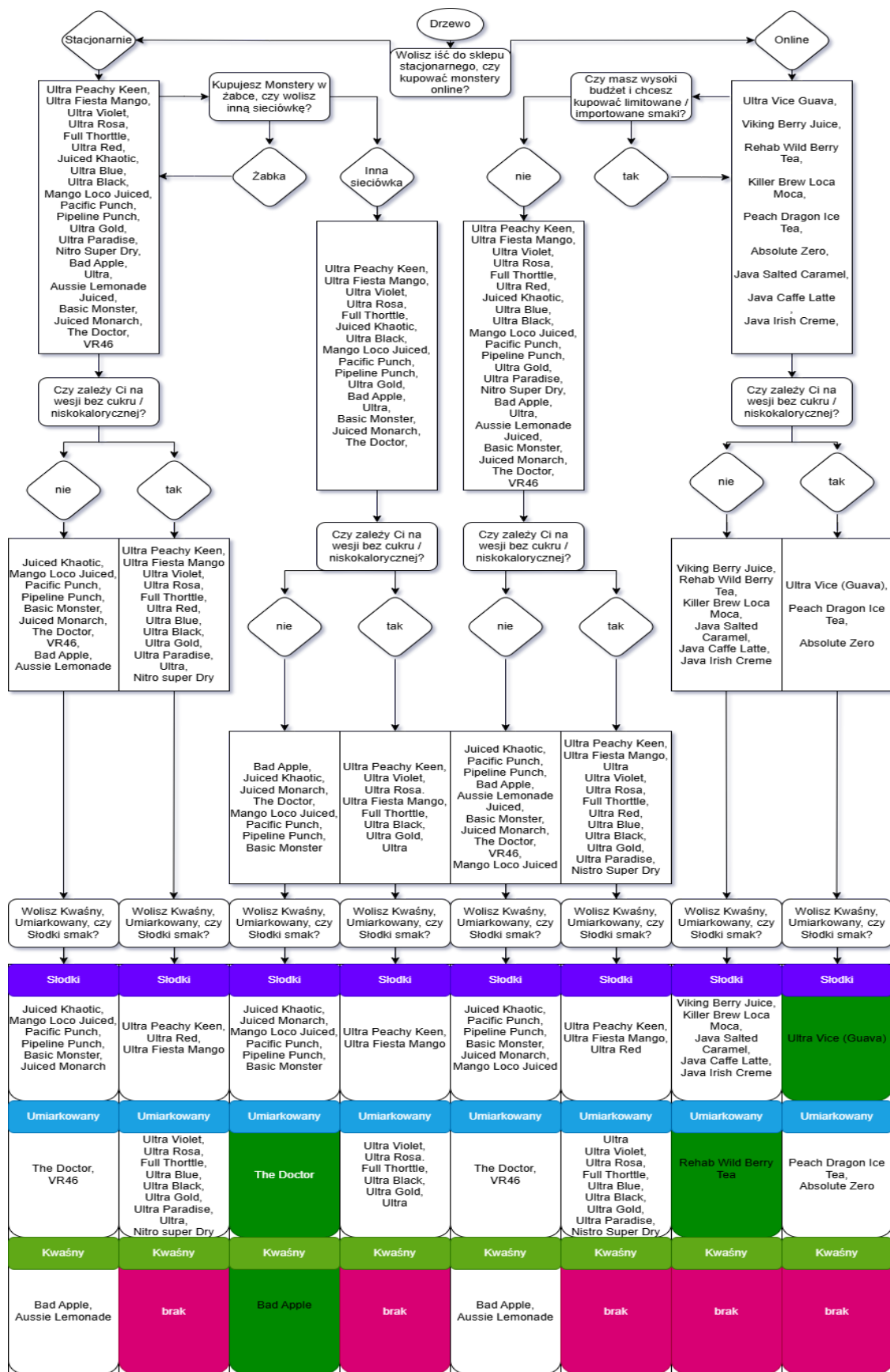
5.4 System Rekomendacyjny (Drzewo Decyzyjne)

Moduł quizu opiera się na deterministycznym drzewie decyzyjnym zdefiniowanym w formacie JSON.

- **Przetwarzanie regułowe:** Zastosowanie operacji na zbiorach (intersection) pozwala na błyskawiczne filtrowanie bazy produktów (smaków) w oparciu o kryteria takie jak typ sklepu, budżet czy profil smakowy. Oddzielenie logiki drzewa od kodu pozwala na modyfikację reguł bez ingerencji w strukturę aplikacji.



Rysunek 6: Analiza pokrycia systemu eksperckiego: liczba rekomendacji dla różnych profili użytkowników.



Rysunek 7: Graficzna reprezentacja drzewa decyzyjnego wykorzystywanego w module rekomendacji.

6 Kluczowe fragmenty kodu źródłowego

W celu zaprezentowania technicznej implementacji opisanych wcześniej modeli, poniżej zamieszczono kluczowe fragmenty logiki biznesowej.

6.1 Implementacja Uczenia Transferowego (Transfer Learning)

Poniższy fragment kodu z modułu uczącego udowadnia zastosowanie techniki Transfer Learningu. Oryginalne wagi modelu pre-trenowanego na zbiorze ImageNet (np. ResNet18) zostają zamrożone, a optymalizacji podlega jedynie nowo zainicjowana warstwa w pełni połączona (`nn.Linear`), dostosowana do liczby klas decyzyjnych w systemie.

```
def get_model(name, n_classes, pretrained=True):
    if name == "resnet18":
        model = models.resnet18(weights='DEFAULT' if pretrained else None)
        for param in model.parameters():
            param.requires_grad = False

        num_ftrs = model.fc.in_features
        model.fc = nn.Linear(num_ftrs, n_classes)

    elif name == "mobilenet_v2":
        model = models.mobilenet_v2(weights='DEFAULT' if pretrained else None)
        for param in model.parameters():
            param.requires_grad = False
        num_ftrs = model.classifier[1].in_features
        model.classifier[1] = nn.Linear(num_ftrs, n_classes)
    else:
        raise ValueError("Unknown model: " + name)
    return model
```

Listing 1: Funkcja inicjująca architekturę modelu (plik `train_binary.py`)

6.2 Dwuetapowa inferencja modelu wizyjnego w API

Przetwarzanie zapytań od użytkownika oparto na dwuetapowym procesie wnioskowania. Wykorzystanie bloku `torch.no_grad()` optymalizuje pamięć serwera poprzez wyłączenie śledzenia gradientów. Model binarny pełni rolę filtra brzegowego, zabezpieczając system przed błędnymi danymi wejściowymi.

```
def predict(self, img_bytes: bytes) -> PredictionResponse:
    # ... (Transformacja zdjęcia do tensora img_t) ...

    with torch.no_grad():
        bin_out = self.binary_model(img_t)
        if bin_out.dim() == 1:
            bin_out = bin_out.unsqueeze(0)

        bin_probs = torch.nn.functional.softmax(bin_out, dim=1).squeeze()
        bin_conf_val, bin_idx_val = torch.max(bin_probs, dim=0)
        best_binary_label = self.binary_classes[bin_idx_val.item()]

        if best_binary_label == "Not_Monster":
            return PredictionResponse(
                label="This image does not contain a monster.",
                confidence=float(bin_conf_val.item()),
                prediction=[],
                num_classes=len(self.binary_classes)
            )

    out = self.model(img_t)

    if out.dim() == 1:
        out = out.unsqueeze(0)

    probs = torch.nn.functional.softmax(out, dim=1).squeeze()
    conf_val, idx_val = torch.max(probs, dim=0)
    best_label = self.classes[idx_val.item()]
    best_conf = float(conf_val.item())

    # ... (Formatowanie i sortowanie wyników) ...
```

Listing 2: Logika predykcji w serwisie backendowym (plik `prediction_services.py`)

6.3 Funkcja oceny (Fitness) w Algorytmie Genetycznym

Kluczowym elementem optymalizacji harmonogramu jest odpowiednio zaprojektowana funkcja przystosowania. Zaprezentowana poniżej metoda ocenia każdego osobnika (potencjalny plan spożycia), przyznając punkty za pokrycie zaplanowanych zadań (pracy/nauki) oknem działania stymulanta, a jednocześnie nakładając surowe kary matematyczne za zjawisko zbyt częstego spożycia (nakładanie się na siebie dawek).

```
def _calculate_fitness(self, individual: List[Tuple[float, float]],
                        wake_float: float, sleep_float: float,
                        task_intervals: List[Tuple[float, float]]) -> float:
    if not self._is_valid_individual(individual, wake_float, sleep_float):
        return INVALID_FITNESS

    intervals = [(start, start + effect) for start, effect in individual]
    penalty = sum(
        IntervalUtils.overlap_length(interval_a[0], interval_a[1], interval_b[0],
        interval_b[1]) * self.overlap_penalty
        for idx_a, interval_a in enumerate(intervals)
        for interval_b in intervals[idx_a + 1:]
    )
    merged_intervals = IntervalUtils.merge(intervals)
    covered = sum(
        IntervalUtils.overlap_length(interval_start, interval_end, task_start,
        task_end)
        for task_start, task_end in task_intervals
        for interval_start, interval_end in merged_intervals
    )
    return covered - penalty
```

Listing 3: Implementacja funkcji fitness dla harmonogramu (plik `schedule_services.py`)

6.4 Mutacja z wykorzystaniem rozkładu Gaussa

W celu uniknięcia utknięcia algorytmu w minimach lokalnych, zastosowano zaawansowany operator mutacji. W przeciwieństwie do czysto losowych modyfikacji, algorytm wykorzystuje rozkład normalny (Gaussa), co pozwala na ewolucyjne, delikatne przesuwanie godzin spożycia. Metoda ta ściśle pilnuje również twardych ograniczeń fizjologicznych – np. bufora czasowego przed snem (`buffer_hours`).

```
def _mutate(self, individual: List[Tuple[float, float]], wake_float: float,
            sleep_float: float, monster_count: int, mutation_rate: float) -> List[
    Tuple[float, float]]:
    mutated = []
    for i, (start, effect) in enumerate(individual):
        if random.random() < mutation_rate:
            start += random.gauss(0, self.gaussian_std)
        if random.random() < mutation_rate:
            effect += random.gauss(0, self.gaussian_std)

        start = max(wake_float, min(start, sleep_float - self.max_effect))
        effect = max(self.min_effect, min(effect, self.max_effect))

        if i > 0:
            start = max(start, mutated[-1][0] + self.min_gap)
        if start + effect > sleep_float - self.buffer_hours:
            effect = max(self.min_effect, sleep_float - self.buffer_hours - start)

        mutated.append((start, effect))
    return mutated
```

Listing 4: Operator mutacji Gaussa z zachowaniem ograniczeń fizjologicznych (plik `schedule_services.py`)

6.5 Silnik wnioskujący Systemu Eksperckiego

Moduł rekomendacyjny zaimplementowano jako klasyczny system ekspercki oparty na drzewie decyzyjnym. Baza wiedzy (reguły i pytania) została odseparowana od logiki biznesowej i jest przechowywana w strukturze JSON.

Poniższy fragment pliku `tree.json` obrazuje strukturę pojedynczego węzła decyzyjnego. Każda odpowiedź zawiera tablicę `filter`, która określa pulę napojów spełniających dane kryterium, oraz zagnieżdżone kolejne pytanie do użytkownika.

```
{
  "question": "Wolisz iść do sklepu stacjonarnego, czy zamawiać
  Monster online?",
  "answers": {
    "Stacjonarnie": {
      "filter": [
        "Ultra Peachy Keen",
        "Ultra Fiesta Mango",
        "Juiced Khaotic",
        "Pipeline Punch"
      ],
      "question": "Kupujesz Monstery w Żabce, czy wolisz inną sieci
      ówkę?",
      "answers": { ... }
    }
  }
}
```

Listing 5: Fragment bazy wiedzy reprezentującej drzewo decyzyjne (plik `tree.json`)

Aby zoptymalizować proces filtrowania bazy napojów (ok. 30 różnych smaków), silnik wnioskujący zaimplementowany w języku Python nie przeszukuje relacyjnej bazy danych przy każdym pytaniu. Zamiast tego, wykorzystuje wysokowydajne operacje matematyczne na zbiorach (`set.intersection`), co gwarantuje złożoność obliczeniową pozwalającą na natychmiastowe zwrócenie wyniku.

```
def get_answers(self, answers: QuizAnswersRequest) -> list[str]:
    candidates = set(self.monster_list)

    shop_key = "Stacjonarnie" if answers.shop_type == 0 else "Online"
    node = self.tree.get("answers", {}).get(shop_key, {})
    if "filter" in node:
        candidates = candidates.intersection(set(node["filter"]))

    if answers.shop_type == 0:
        sub_key = "Żabka" if answers.is_zabka else "inne"
    else:
        sub_key = "yes" if answers.high_budget else "no"

    node = node.get("answers", {}).get(sub_key, {})
    if "filter" in node:
        candidates = candidates.intersection(set(node["filter"]))

    # ... (kolejne kroki schodzenia w głąb drzewa, np. zawartosc cukru) ...

    # Zwrocenie przefiltrowanej, dopasowanej listy napojow
    return list(candidates)
```

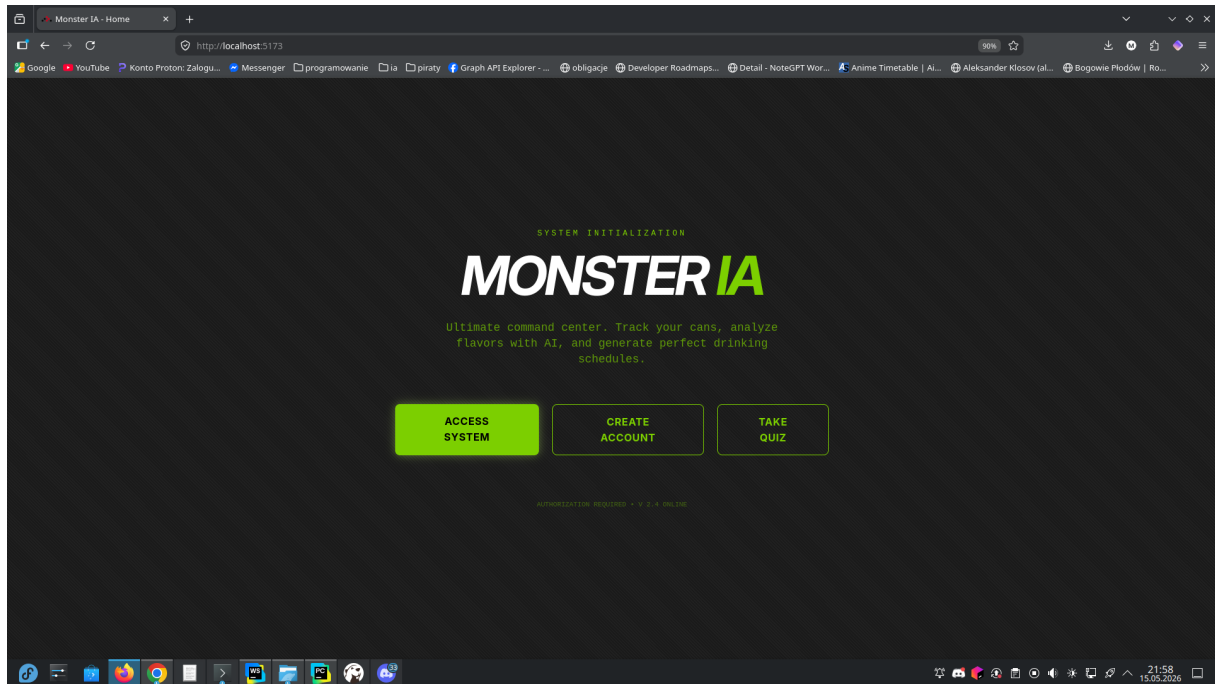
Listing 6: Logika silnika wnioskującego w Pythonie (plik `quiz_services.py`)

7 Prezentacja interfejsu systemu

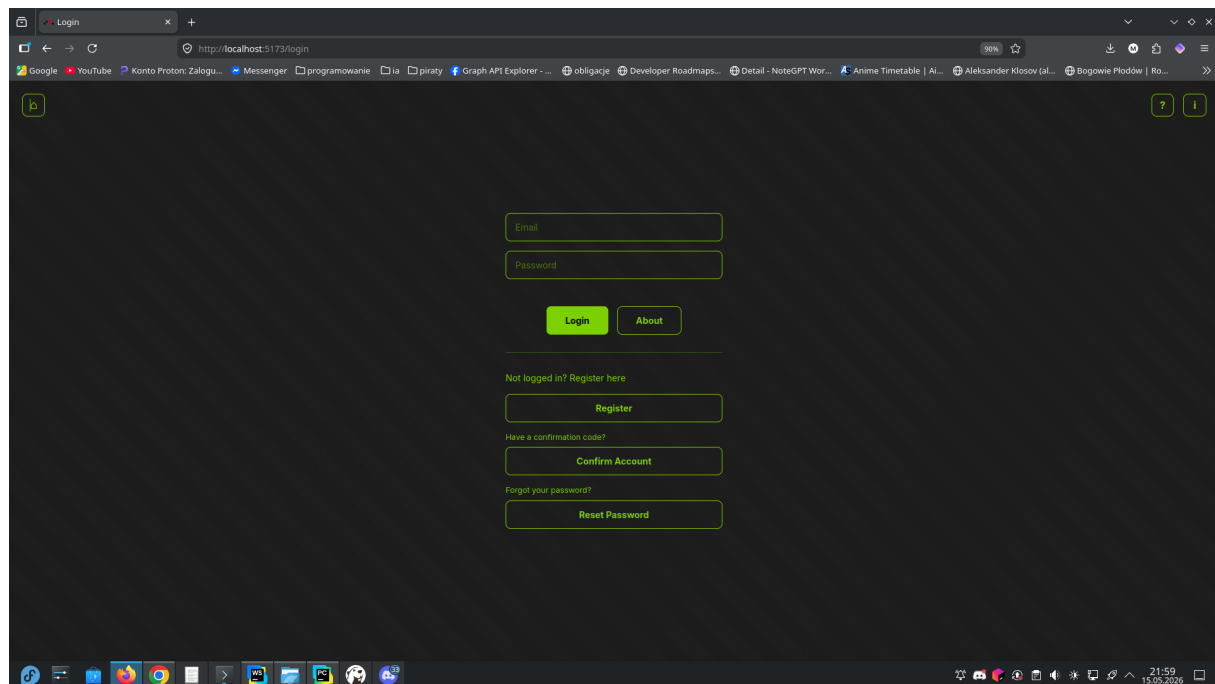
Aby udowodnić poprawne zintegrowanie modułów sztucznej inteligencji z warstwą wizualną i serwerową, poniżej zaprezentowano zrzuty ekranu z działającego systemu.

7.1 Aplikacja Frontend (Widok Użytkownika)

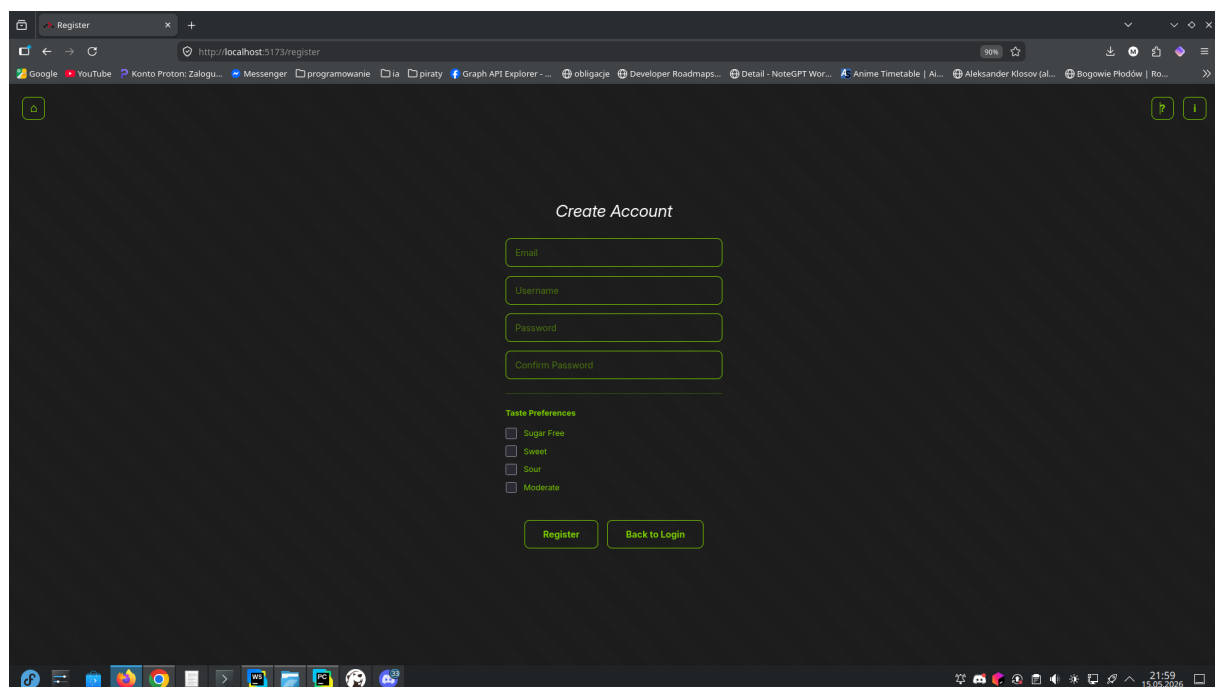
To są przykładowe widoki z aplikacji



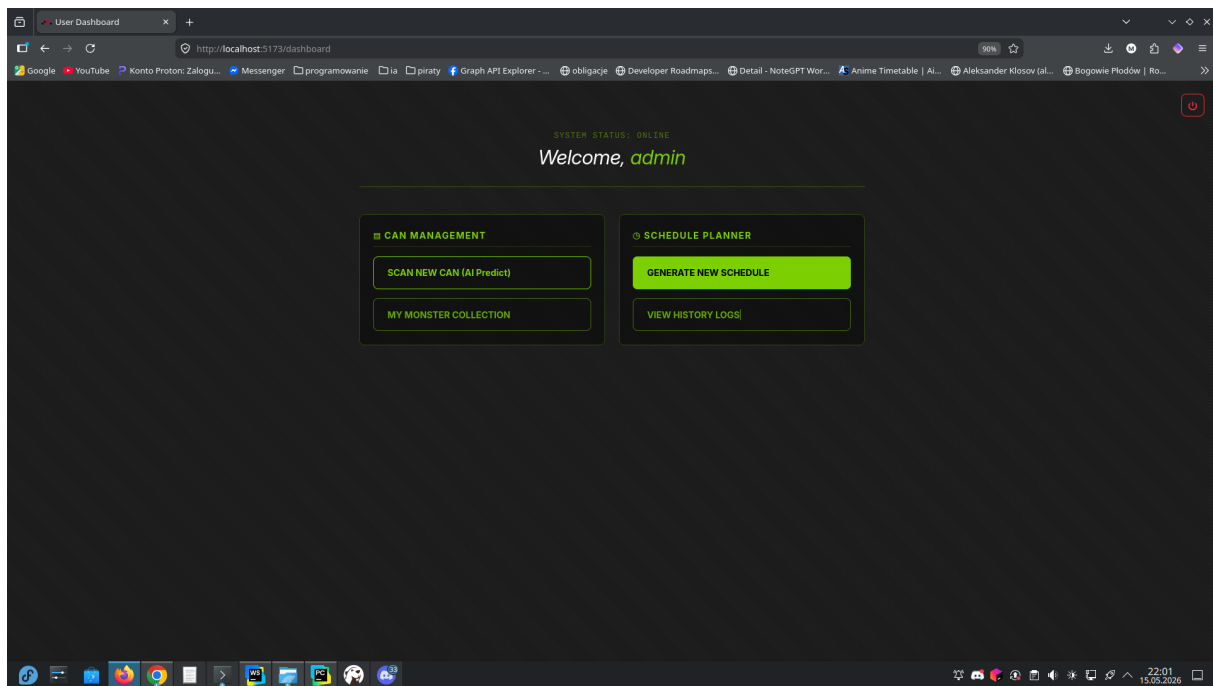
Rysunek 8: Strona główna



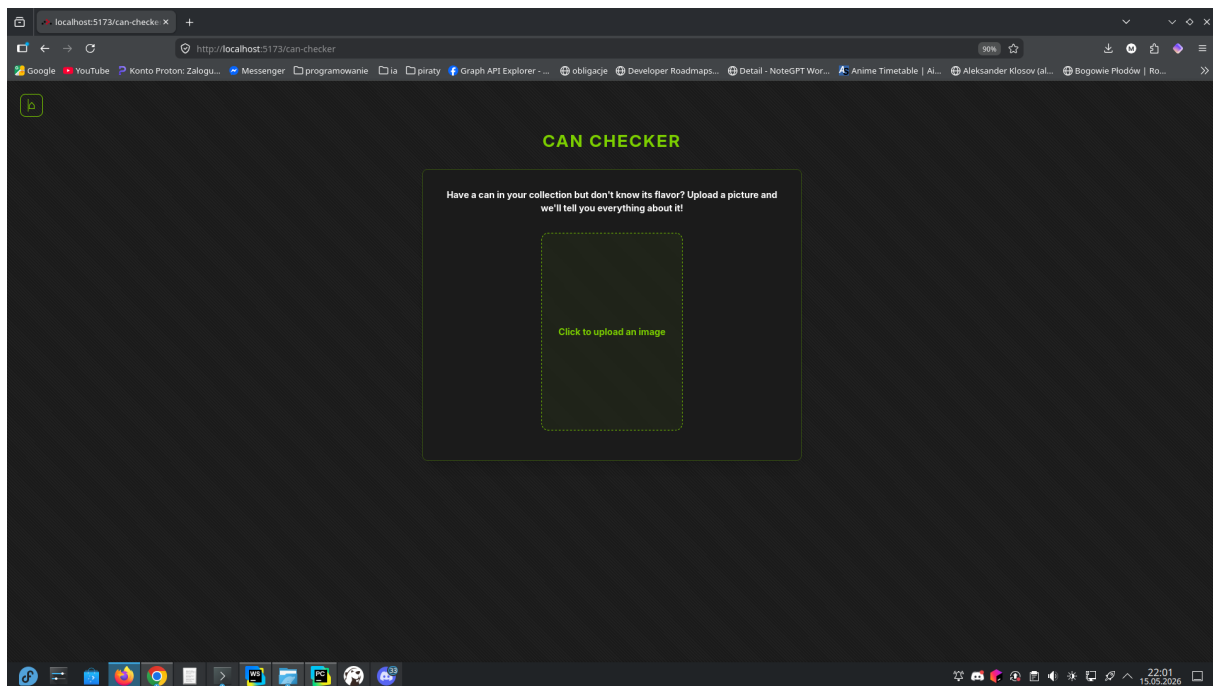
Rysunek 9: Strona logowania



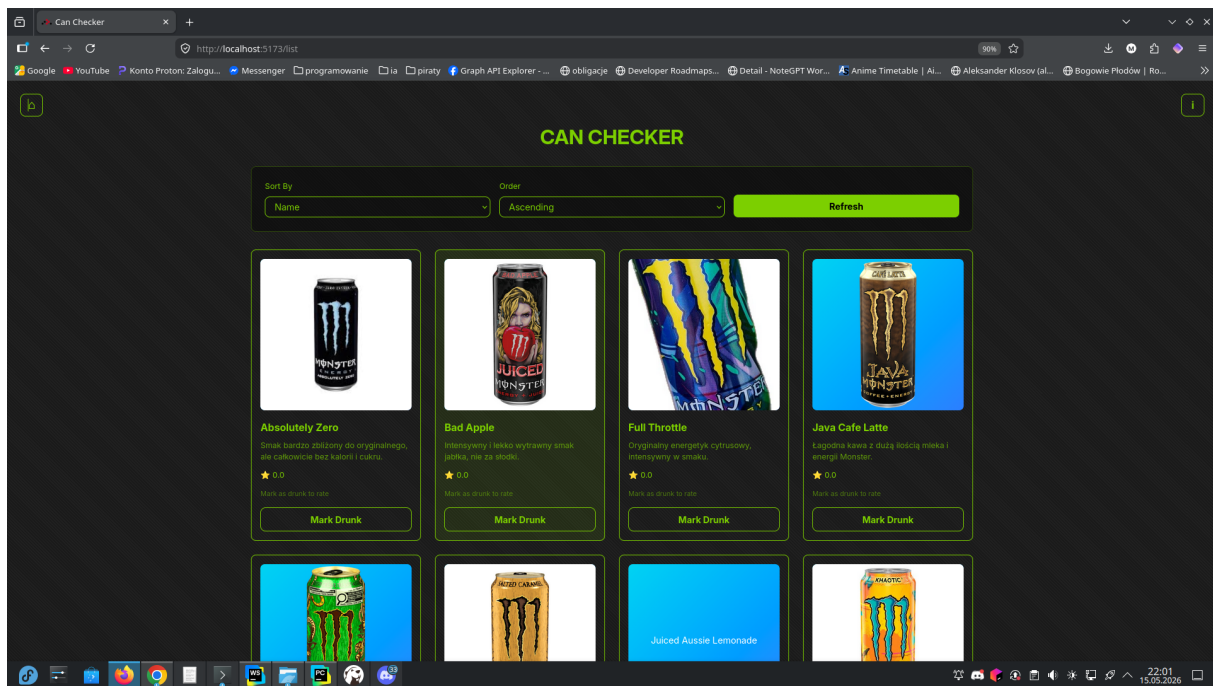
Rysunek 10: Strona rejstracji



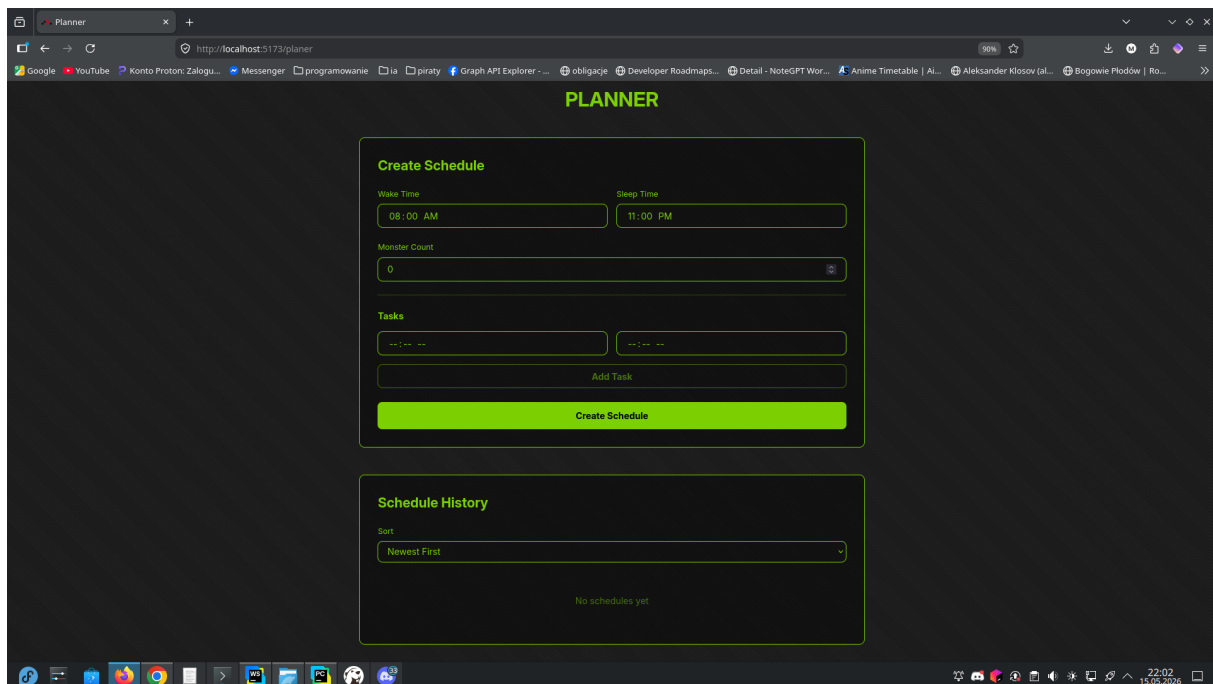
Rysunek 11: Panel użytkownika



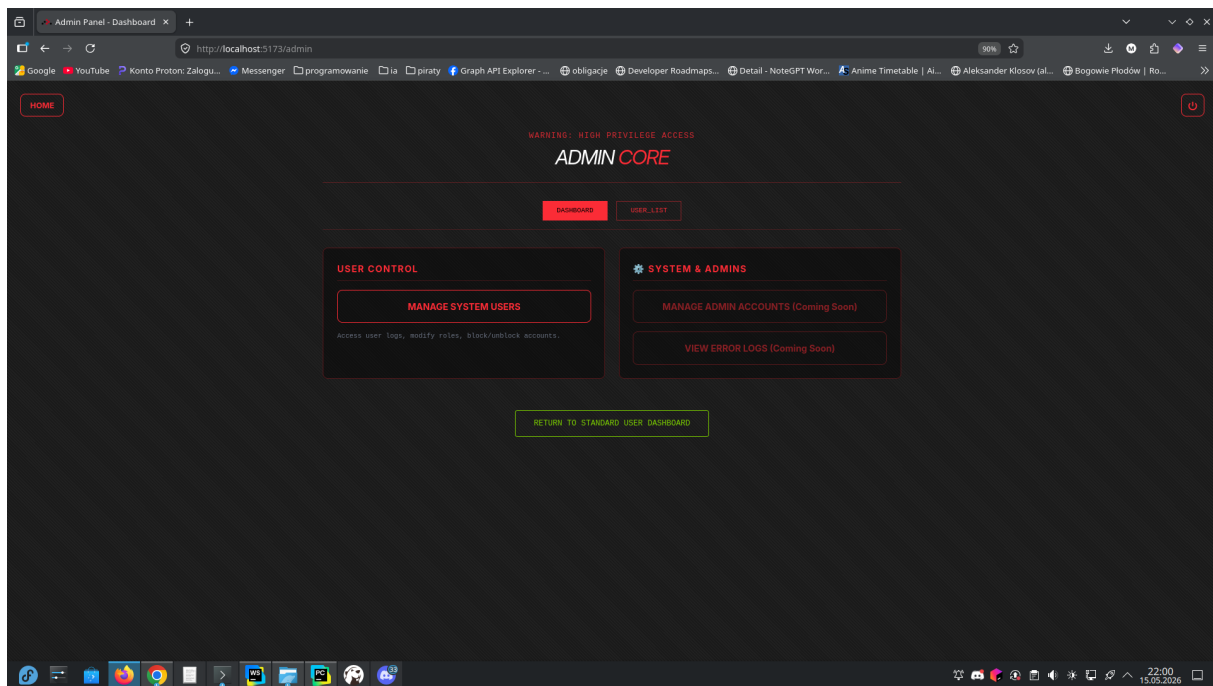
Rysunek 12: Strona do przesyłania zdjęcia puszki do predykcji



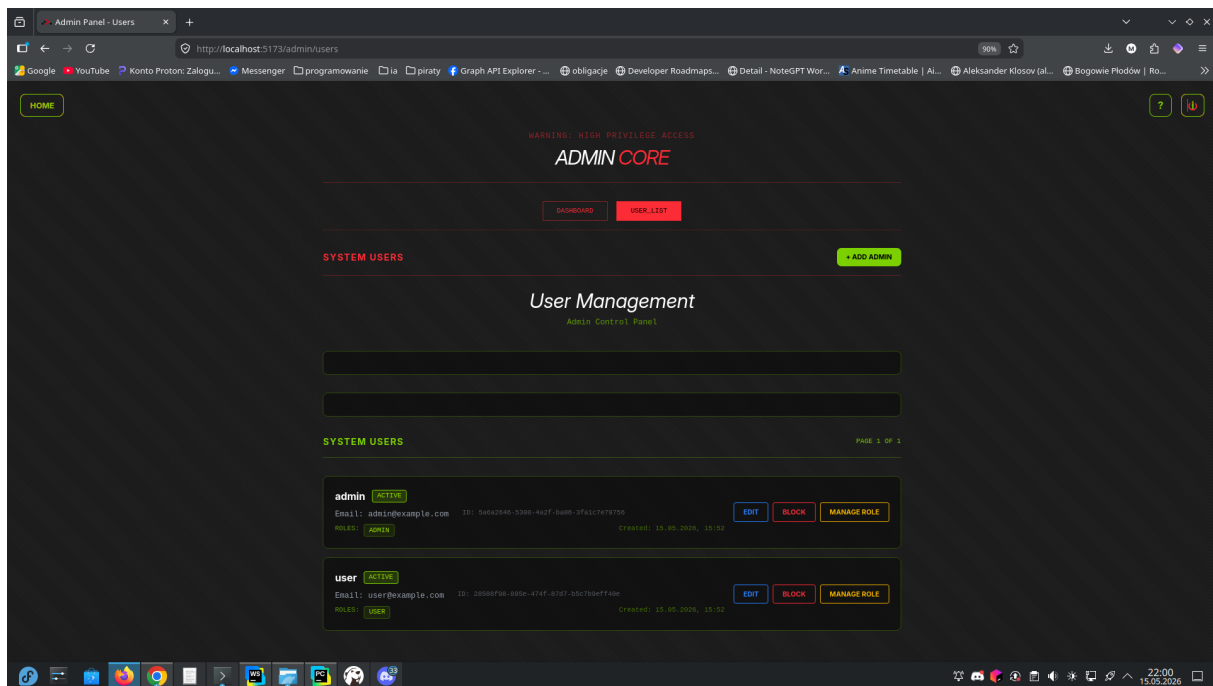
Rysunek 13: Strona listy monsterów



Rysunek 14: Strona generatora planera

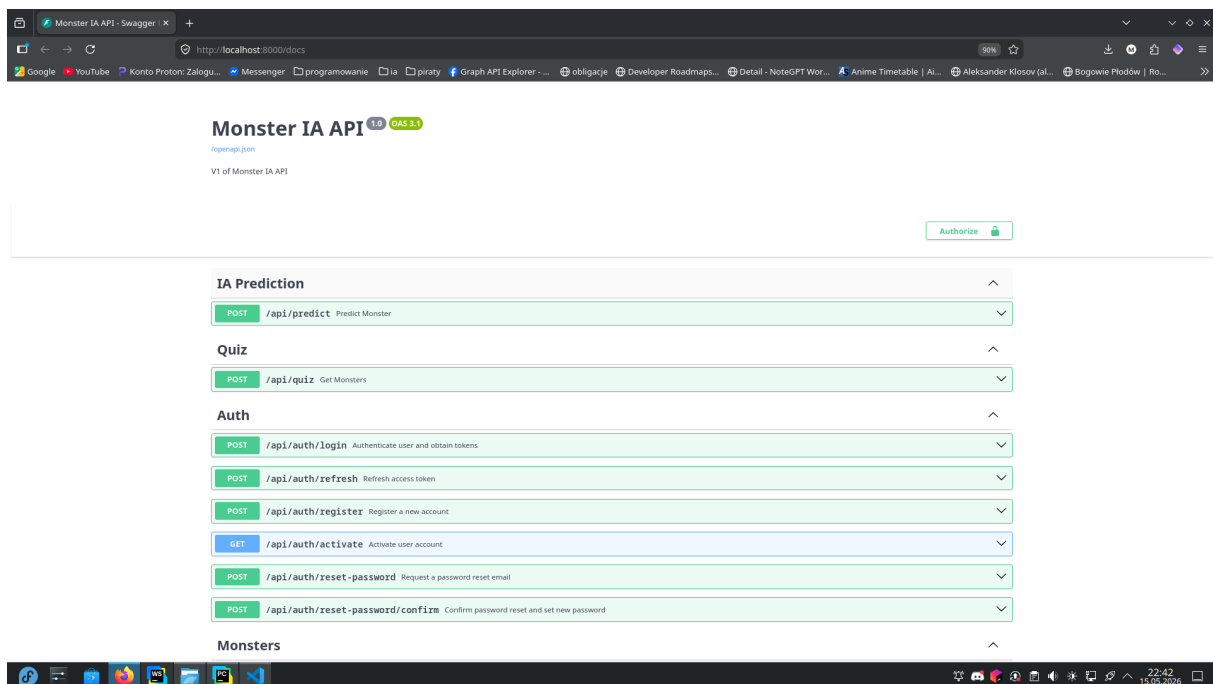


Rysunek 15: Panel administratora

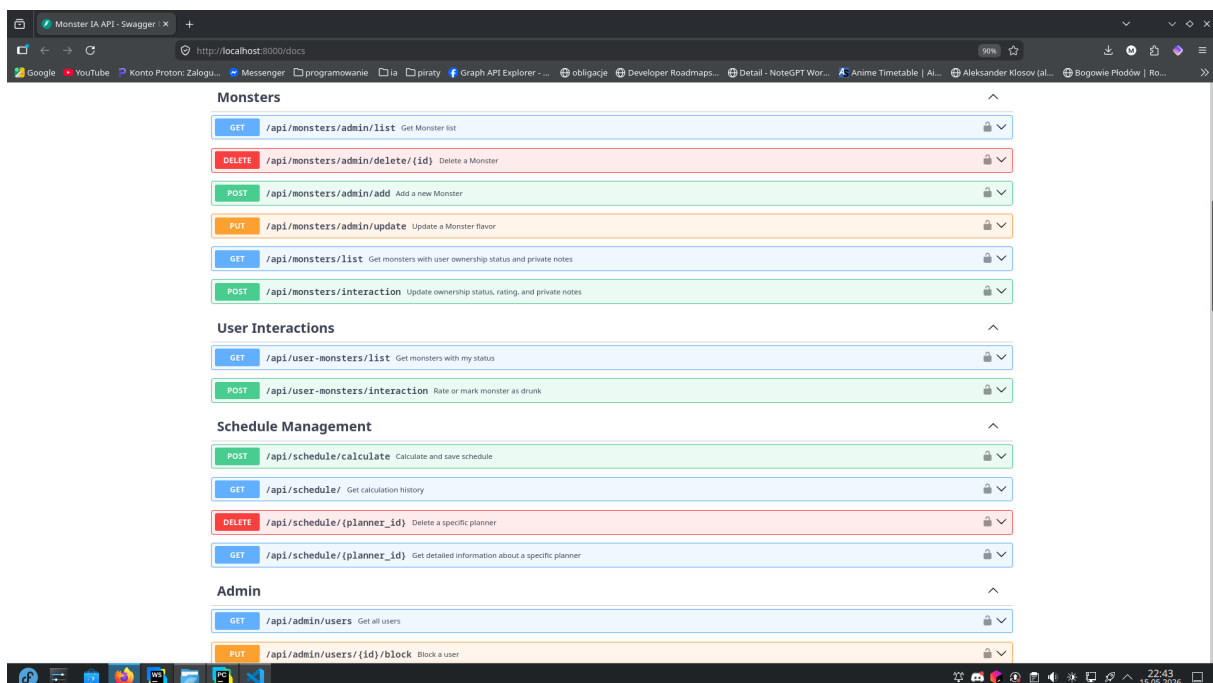


Rysunek 16: Zarządzanie użytkownikami

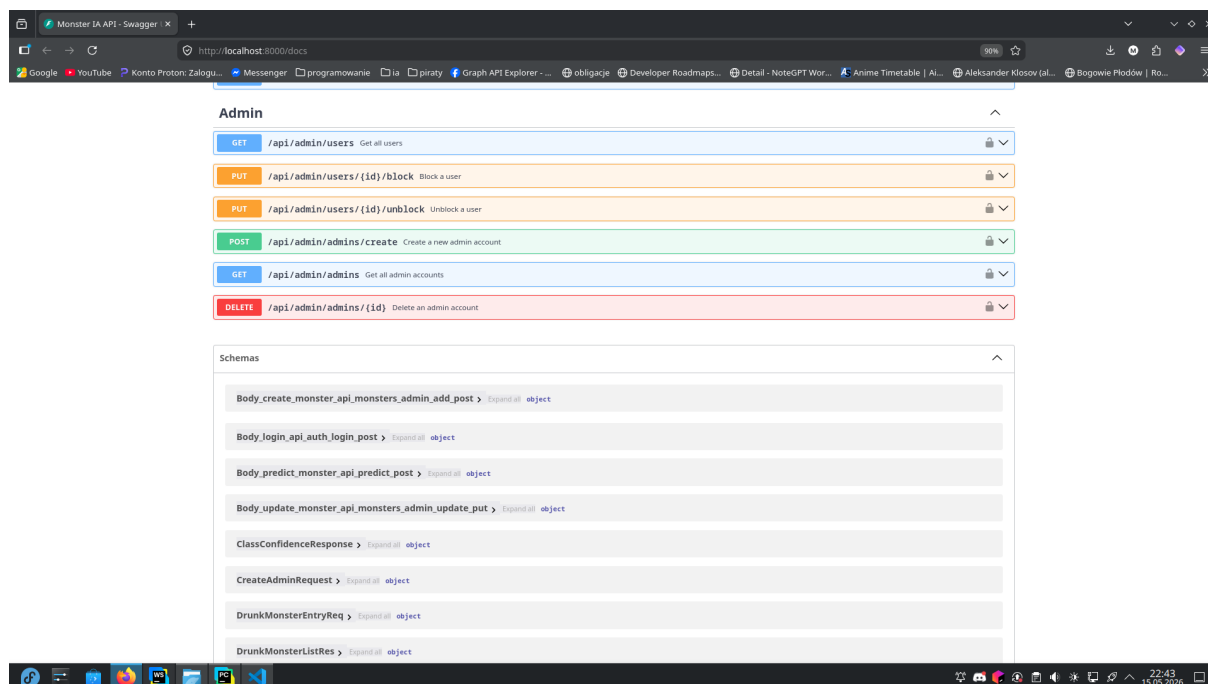
7.2 Aplikacja Backend (Dokumentacja API Swagger)



Rysunek 17: Interaktywna dokumentacja REST API generowana przez framework FastAPI cz1.



Rysunek 18: Interaktywna dokumentacja REST API generowana przez framework FastAPI cz2.



Rysunek 19: Interaktywna dokumentacja REST API generowana przez framework FastAPI cz3.

8 Podsumowanie i wnioski ogólne

Projekt MonsterAI z powodzeniem demonstduje praktyczne wykorzystanie trzech odmien-nych metod sztucznej inteligencji: wizji komputerowej, algorytmów ewolucyjnych i syste-mów eksperckich, zamykając je w jednym, spójnym środowisku webowym. Technologia konteneryzacji Docker oraz asynchroniczność frameworka FastAPI sprawiły, że system jest nie tylko wydajny, ale i prosty we wdrożeniu. Zderzenie wymagań akademickich z komer-cyjnym podejściem do budowy oprogramowania (separacja frontend/backend, chmura S3) okazało się dużym sukcesem zespołu.

9 Wkład, udział i wnioski osobiste

Poniżej znajduje się podsumowanie zaangażowania każdego z członków zespołu, weryfikujące zdobyte umiejętności oraz napotkane trudności.

9.1 Mateusz Bogacz-Drewniak

Zakres prac:

- Zarządzanie projektem
- Przygotowanie środowiska docker
- Implementacja bazy danych i zewnętrznych serwisów (MinIO, MailPit)
- Stworzenie systemu logowania, rejestracji, odzyskiwania hasła i endpointów CRUD

Plusy projektu: Zdobycie praktycznej wiedzy o Transfer Learningu; integracja modeli z API.

Minusy/Trudności: (np. Problemy z doбором odpowiednich parametrów optymalizatora).

9.2 Mateusz Chimkowski

Zakres prac:

Plusy projektu:

Minusy/Trudności:

9.3 Szymon Mikołajek

Zakres prac: Frontend w React, budowa UI i UX, podłączenie API

Plusy projektu:

Minusy/Trudności:

9.4 Paweł Kruk

Zakres prac: Frontend w React, budowa UI i UX, podłączenie API

Plusy projektu: Usprawnienie tworzenia komponentów w Tailwind CSS; nowoczesny routing.

Minusy/Trudności:

9.5 Bartosz Dobroś

Zakres prac:

Plusy projektu:

Minusy/Trudności:

Spis rysunków

1	Diagram związków encji (ERD) przedstawiający strukturę bazy danych systemu.	7
2	Diagram przepływu danych (DFD)	8
3	Pewność i trafność predykcji modelu wizyjnego na zbiorze weryfikacyjnym.	10
4	Histogram rozkładu funkcji Fitness wykazujący wysoką zbieżność algorytmu genetycznego.	11
5	Diagram wpływu zapracowania (liczby zadań) na efektywność wygenerowanego planu.	11
6	Analiza pokrycia systemu eksperckiego: liczba rekomendacji dla różnych profili użytkowników.	12
7	Graficzna reprezentacja drzewa decyzyjnego wykorzystywanego w module rekomendacji.	13
8	Strona główna	18
9	Strona logowania	19
10	Strona rejestracji	19
11	Panel użytkownika	20
12	Strona do przesyłania zdjęcia puszki do predykcji	20
13	Strona listy monsterów	21
14	Strona generatora planera	21
15	Panel administratora	22
16	Zarządzanie użytkownikami	22
17	Interaktywna dokumentacja REST API generowana przez framework FastAPI cz1.	23
18	Interaktywna dokumentacja REST API generowana przez framework FastAPI cz2.	23
19	Interaktywna dokumentacja REST API generowana przez framework FastAPI cz3.	24

Spis tabel

1	Technologie backendowe wykorzystane w projekcie	6
2	Technologie frontendowe wykorzystane w projekcie	6
3	Technologie infrastrukturalne wykorzystane w projekcie	6

Spis fragmentów kodu

1	Funkcja inicjująca architekturę modelu (plik train_binary.py)	14
2	Logika predykcji w serwisie backendowym (plik prediction_services.py)	15
3	Implementacja funkcji fitness dla harmonogramu (plik schedule_services.py)	16
4	Operator mutacji Gaussa z zachowaniem ograniczeń fizjologicznych (plik schedule_services.py)	16

5	Fragment bazy wiedzy reprezentującej drzewo decyzyjne (plik tree.json) . .	17
6	Logika silnika wnioskującego w Pythonie (plik quiz_services.py)	17